
Mantenimiento de Software

Proyecto Final

Mantenimiento de Software

Sistema MercadoVentas

Integrantes:

Hiram Chavez Torres

Leslie Michell Zamarripa Guerrero

*Documento de evidencias del proyecto de refactorización
y mejora del sistema de carrito de compras MercadoVentas*

1. Descripción del Funcionamiento del Sistema

MercadoVentas es un sistema de gestión de ventas desarrollado en Python que opera mediante interfaz de línea de comandos (CLI). El sistema permite la autenticación de usuarios (compradores) y empleados (administradores), la gestión de inventario de productos, la navegación por categorías, la creación de carritos de compra, el procesamiento de pagos con cálculo de cambio, y la generación de tickets de compra en formato TXT.

Actores del Sistema

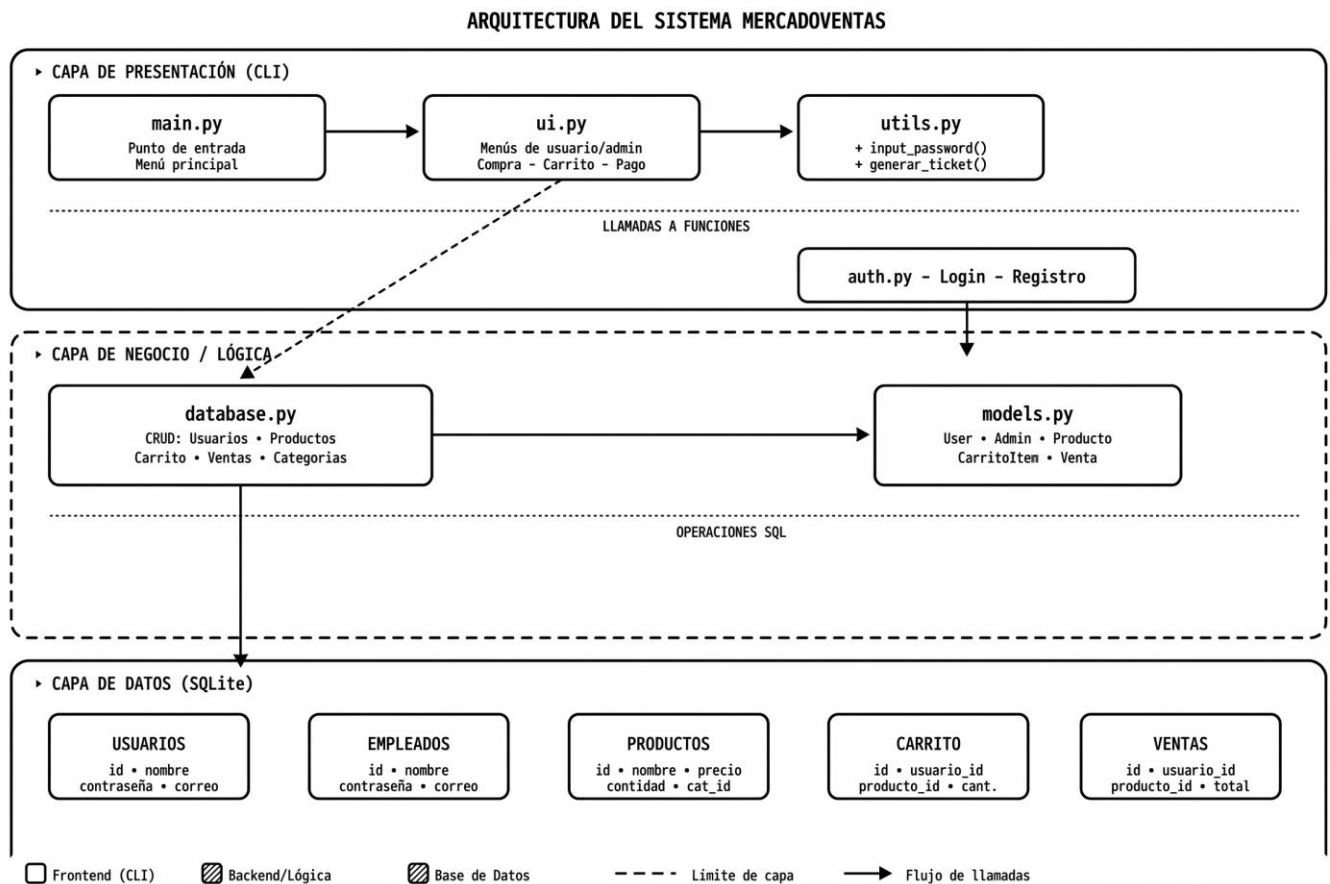
Usuario Comprador: Se registra, inicia sesión, navega categorías, agrega productos al carrito, realiza pagos.

Administrador/Empleado: Gestiona inventario (CRUD de productos), categorías, y revisa el registro de ventas.

Flujo Principal

1. El sistema inicia mostrando el menú principal (login/registro/salir).
2. El usuario puede registrarse como comprador o empleado (datos almacenados con bcrypt).
3. Al iniciar sesión, se valida contra la BD y se redirige según el rol.
4. Los compradores navegan categorías y productos, los agregan al carrito.
5. En el carrito, pueden ver ítems, eliminarlos o proceder al pago.
6. El pago solicita el monto, calcula el cambio y genera un ticket TXT.
7. Los administradores gestionan inventario, categorías y revisan ventas.
8. Toda la información persiste en una base de datos SQLite.

Diagrama de Arquitectura



2. Dificultades Encontradas

Durante el proceso de ingeniería inversa y refactorización se identificaron las siguientes dificultades:

2.1 Código Monolítico sin Documentación

El código original constaba de un único archivo (`main.py`) de 343 líneas sin comentarios ni documentación. Comprender la lógica requirió leer el código completo múltiples veces y mapear mentalmente las funciones y sus dependencias.

2.2 Bugs Críticos en el Login

La función `login()` tenía un error crítico: al usar `fetchone()` para obtener el nombre del usuario, siempre devolvía el primer registro de la tabla en lugar del usuario autenticado. Esto significaba que cualquier usuario podía iniciar sesión y ver los datos de otro.

2.3 Errores de Parámetros en Llamadas a Funciones

Varias funciones recibían los parámetros en orden incorrecto (por ejemplo, `Categorias()` recibía `cursorDB`, `conexion`, `name`, `userID` en lugar del orden esperado). Esto causaba fallos aleatorios difíciles de depurar.

2.4 Manejo Incorrecto de Tipos de Datos

En varias consultas SQL, los valores se pasaban sin la tupla correcta (e.g., `(select)` en lugar de `(select,)`), lo que causaba errores silenciosos en `DELETE` e `INSERT`. Además, `userID` se trataba como entero cuando era una tupla.

2.5 Ventana de Tiempo Ajustada

La entrega del proyecto coincidió con otras materias, por lo que se requirió una planificación cuidadosa para completar todas las actividades en el tiempo disponible.

3. Metodos Utilizados para Entender elCodigo

3.1 Lectura Sistemática del Código

Se realizó una lectura completa del archivo main.py (343 líneas) identificando: estructuras de datos, flujo de control, conexiones a BD, y funciones principales. Se tomaron notas de cada función.

3.2 Trazado de Dependencias

Se mapearon las llamadas entre funciones para entender el flujo de ejecución. Se identificaron las dependencias: menu() -> login() -> InterfazU/Interfaz -> compra/venta/Inventario -> database.

3.3 Inspección de la Base de Datos

Se abrió la base de datos SQLite (DataBaseMercado.db) para entender el esquema de tablas, relaciones y datos existentes. Se identificaron 6 tablas: USUARIOS, EMPLEADOS, PRODUCTOS, CATEGORIA, Carrito_Compras, VENTAS.

3.4 Ejecución de Pruebas Exploratorias

Se ejecutó el sistema varias veces para probar diferentes flujos (registro, login erróneo, compra, pago) y observar comportamientos inesperados. Esto permitió descubrir bugs que no eran obvios solo con la lectura.

3.5 Análisis de Datos Existentes

Se ejecutaron consultas directamente en la BD para entender que datos existían y validar que el sistema funcionara correctamente con los datos reales.

4. Clasificacion de Cambios Realizados

4.1 Mantenimiento Correctivo

Correccion de errores y defectos existentes en el sistema original:

- Bug en login(): se usaba fetchone() sin filtrar por el usuario autenticado. Correccion: buscar por correo.
- Bug en menu() opcion empleado: se comparaba opcion en vez de opcion1. Correccion: variable correcta.
- Parametros en orden incorrecto en Categorias(). Correccion: orden de parametros corregido.
- DELETE FROM PRODUCTOS sin tupla correcta (select vs (select,)). Correccion: sintaxis SQL corregida.
- userID tratado como entero siendo tupla en DELETE carrito. Correccion: userID[0].
- Registro de venta: solo guardaba el ultimo producto. Correccion: registro por cada item.

4.2 Mantenimiento Preventivo

Acciones para prevenir problemas futuros y mejorar la mantenibilidad:

- Se agrego .gitignore para excluir archivos temporales, .db, __pycache__.
- Se agregaron try/except en todas las funciones para manejo de errores.
- Se agrego manejo de KeyboardInterrupt en main.py.
- Se crearon pruebas automatizadas (test_sistema.py) que verifican el flujo completo.
- Se valido entrada de datos en precio, cantidad e IDs para evitar errores de tipo.

4.3 Mantenimiento Perfectivo

Mejoras en la estructura y calidad del codigo sin cambiar la funcionalidad externa:

- Separacion del codigo monolítico en 6 modulos: models, database, auth, ui, utils, main.
- Creacion de clases de dominio: User, Admin, Producto, CarritoItem, Venta.
- Encapsulamiento de todas las consultas SQL en database.py (patron Repository).
- Docstrings en todos los modulos y funciones principales.
- Nombres de funciones y variables en ingles (estandar en desarrollo de software).
- Eliminacion de codigo muerto (clase Carrito sin uso, variable Incremento).
- Organizacion en capas: presentacion (ui.py), negocio (auth.py, database.py), datos (SQLite).

4.4 Mantenimiento Adaptativo

Cambios para adaptarse a nuevos requerimientos o entornos:

- Input de contrasena con mascara (): se implemento input_password() con lectura caracter por caracter.
- Nueva funcionalidad de pago: solicitud de monto, validacion contra total, calculo de cambio.
- Generacion de ticket de compra en formato TXT con: fecha, productos, cantidades, subtotales, total, pago, cambio.
- Refactor de menus con bucles while (en lugar de recursion) para mejor flujo de navegacion.
- Soporte multiplataforma: input_password() funciona tanto en Windows (msvcrt) como Unix (termios).

5. Capturas del Sistema Funcionando

5.1 Menu Principal

Al ejecutar el sistema, se muestra el menu principal con opciones de login, registro y salida.

[Menu principal con opciones: 1. Iniciar sesion, 2. Crear cuenta, 3. Salir]

5.2 Registro de Usuario

El usuario puede registrarse con nombre, contrasena (con mascara), correo y telefono.

[Registro: ingreso de datos con contrasena mostrando simbolos]

5.3 Inicio de Sesion

Login validando credenciales contra la base de datos usando bcrypt.

[Inicio de sesion exitoso mostrando nombre del usuario]

5.4 Menu de Usuario Comprador

Menu con opciones: Comprar producto, Ver/Pagar carrito, Salir.

[Interfaz de usuario comprador con 3 opciones]

5.5 Navegacion de Categorias y Productos

El usuario navega categorias, selecciona una y visualiza los productos disponibles.

[Seleccion de categoria y lista de productos con precios y stock]

5.6 Carrito de Compras

Visualizacion del carrito con productos, cantidades, subtotales y total.

[Carrito mostrando items, precios y total a pagar]

5.7 Proceso de Pago

Pago con solicitud de monto, validacion y calculo de cambio.

[Proceso de pago: total, monto ingresado, cambio calculado]

5.8 Ticket de Compra TXT

Ticket generado automaticamente al confirmar el pago.

[Archivo ticket_[usuario]_[fecha].txt con detalle de compra]

5.9 Menu de Administrador

Menu con opciones: Gestion de inventarios, Ver categorias, Registro de ventas.

[Interfaz de empleado con CRUD de productos y categorias]

6. Capturas de Commits y Ramas

6.1 Historial de Commits

El proyecto se gestiona con Git desde el inicio. Se crearon las siguientes ramas:

- master: rama principal con el código estable.
- refactor: rama de trabajo para la reestructuración y mejoras.

Commits realizados:

- commit 1: chore: commit inicial - sistema original MercadoVentas (main.py monolítico)
- commit 2: refactor: separar código monolítico en módulos (models, database, auth, ui, utils)
- commit 3: feat: agregar test_sistema.py y corrección en validación de ticket
- commit 4: chore: .gitignore para tickets y limpiar archivos temporales
- commit 5: merge: refactor completo del sistema MercadoVentas (merge a master)

6.2 Proceso de Merge

Se utilizó la estrategia merge --no-ff para mantener un historial claro. La rama refactor se fusionó con master una vez completados y verificados todos los cambios.

6.3 Comandos Git Utilizados

```
git init
git add -A
git commit -m "mensaje"
git checkout -b refactor
git checkout master
git merge refactor --no-ff
git log --oneline --graph
```

7. Link del Repositorio GitHub

[HiramCT/MercadoVentas-MantenimientoSoftware](#)

8. Conclusiones

El sistema MercadoVentas permitio aplicar los conceptos fundamentales de la materia: ingenieria inversa, refactorizacion, y los cuatro tipos de mantenimiento (correctivo, preventivo, perfectivo y adaptativo).

La ingenieria inversa fue un paso crucial: antes de cualquier modificacion, fue necesario comprender a fondo el codigo existente, sus deficiencias y su logica de negocio. Esto permitio identificar mas de 10 bugs que habrian pasado desapercibidos sin un analisis cuidadoso.

La refactorizacion del monolitico a una arquitectura modular (6 modulos con responsabilidades claras) mejoro significativamente la mantenibilidad del codigo. Ahora es mas facil entender, probar y extender cada componente de forma independiente.

Las nuevas funcionalidades (pago con calculo de cambio, ticket TXT, contraseña con mascara) no solo cumplen con los requisitos del proyecto, sino que mejoran la experiencia de usuario y la seguridad del sistema.

En conclusion, este proyecto evidencia que el mantenimiento de software es una actividad fundamental en el ciclo de vida de cualquier sistema. No se trata solo de "arreglar lo que esta roto", sino de mejorar continuamente la calidad, estructura y funcionalidad del software para adaptarlo a las necesidades cambiantes de los usuarios y del negocio.